

DATA 201 · MID-PROJECT PRESENTATION · GROUP 7

Olist Brazilian E-Commerce

Sathwika Roy Tulasi · Aimaan Khan · Drumil Patil · Ellie Le



Open our GitHub repository

github.com/SathwikaRoy2002/BrazilianE-Commerce_group7_data201

1.55M

raw rows in 9 CSV
files

11

tables in 3NF, 11
foreign keys

16

SQL queries: 8 basic
+ 8 advanced

October 2026

[Dataset](#)[Schema](#)[SQL](#)[Next steps](#)

Agenda: click a section to jump

01

Dataset

- Source, size, 52 columns
- Why we chose Olist
- Messiness and cleaning needs

≈ 1:20



02

Schema & setup

- ER diagram → 11 tables
- Keys and relationships
- 1NF → 3NF
- Import evidence

≈ 2:30



03

SQL exploration

- Each member explains one advanced query
- All 2 basic + 2 advanced listed
- Issues while querying

≈ 3:50



04

Next steps

- Difficulties so far
- Plan for the final
- Team and GitHub

≈ 0:40



Q&A backup full SQL for all 16 queries, the FD tests behind our 3NF claims, and the MySQL load run

[Open ›](#)



Olist: 9 linked CSV files, 1.55M rows

1,550,922

rows across 9 CSV files (121 MB)

52

columns (features) across the 9 files

99,441

orders placed Sep 2016 – Oct 2018

Source: Kaggle, “Brazilian E-Commerce Public Dataset by Olist”, real anonymized orders from Olist, a Brazilian marketplace that connects small sellers to large online stores.

File (olist_...)	Rows	Cols	What it holds
customers	99,441	5	customer id, zip, city, state
geolocation	1,000,163	5	lat/lng points per zip prefix
order_items	112,650	7	one row per unit sold
order_payments	103,886	5	type, installments, value
order_reviews	99,224	7	score 1–5, title, comment
orders	99,441	8	status + 5 timestamps
products	32,951	9	category, size, weight, photos
sellers	3,095	4	seller zip, city, state
category_translation	71	2	Portuguese → English names
Total	1,550,922	52	



Why Olist, and why it isn't a toy dataset

Why we chose it



Truly relational

The 9 files share 6 keys (order, customer, product, seller, zip, category), so primary and foreign keys come from the data itself.



Needs normalization

Duplicate rows, a review key repeated across orders, and zip → state dependencies make 1NF → 3NF a real exercise.



Rich enough to query

8 timestamps, categories (status, payment, state, product), numbers (price, freight, weight), geography and review text.

What makes it messy (cleaning needs)

261,831

exact duplicate rows in geolocation, up to 1,146 rows for one zip

789

review ids attached to more than one order

610

products with no category; 2 categories missing from the translation file

23

seller-city spellings merged ("sbc/sp", "lages - sc"); an e-mail and a zip typed as a city

2,965

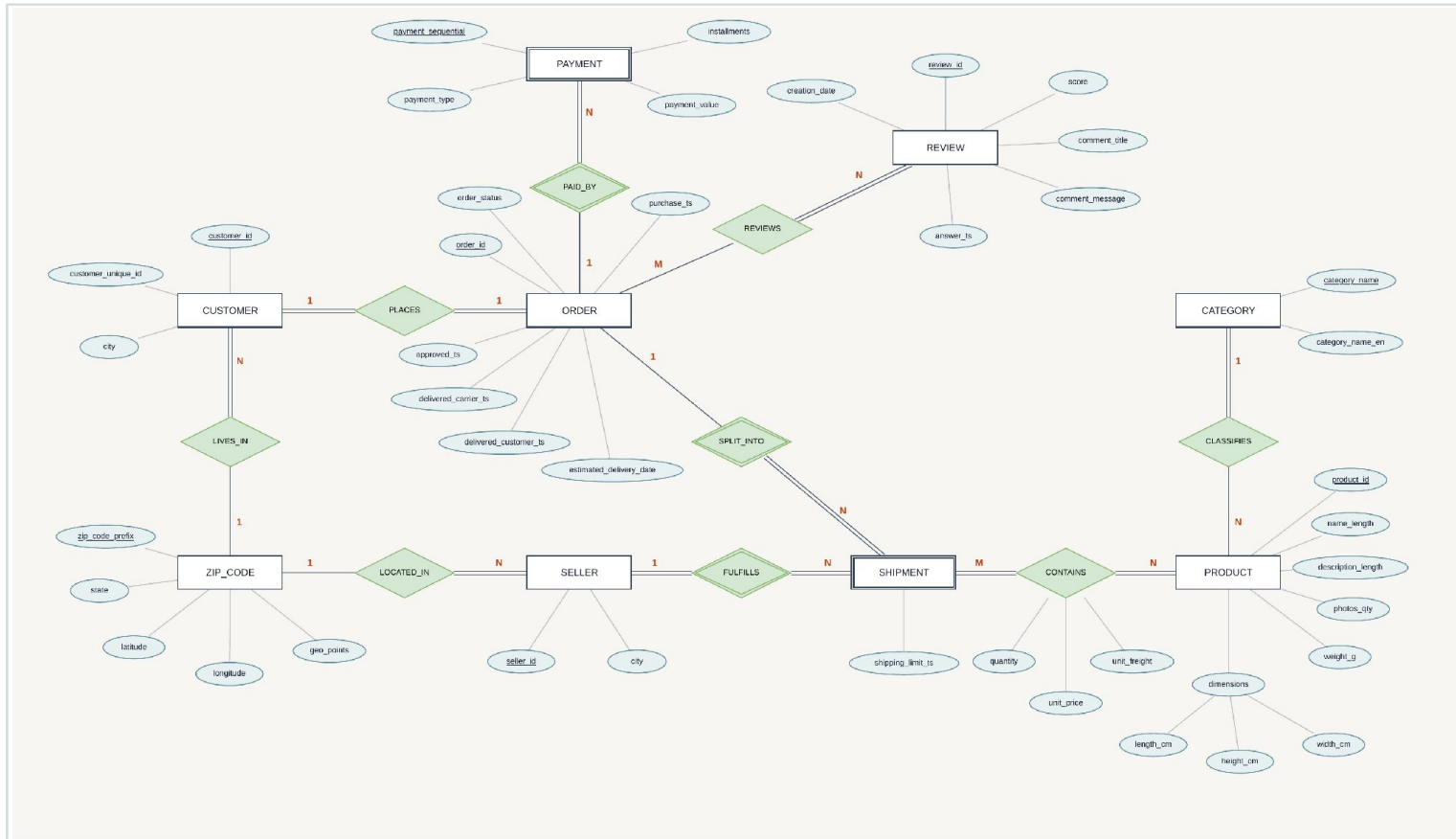
orders whose delivery date is an empty string, not NULL

3

import traps: CRLF line endings, a byte-order mark, and a review ending in "\n" that merged two rows

[Dataset](#)[Schema](#)[SQL](#)[Next steps](#)

ER diagram



docs/olist_er_diagram.drawio.pdf · Chen notation: double box = weak entity, double diamond = identifying relationship

How the ER maps to tables

7 strong entities → 7 tables with their own key

Weak PAYMENT → PK (order_id, payment_sequential)

Weak SHIPMENT (owners ORDER + SELLER) → PK (order_id, seller_id)

M:N CONTAINS → order_item with quantity, unit_price, unit_freight

M:N REVIEWS → bridge table order_review

1:N links → FK on the N side; composite “dimensions” split into 3 columns

= 11 tables



Relational schema: 11 tables

Table	Primary key	Foreign keys	Why
zip_code	zip_code_prefix	—	state and coordinates depend only on the zip
customer	customer_id	zip_code_prefix → zip_code	Olist issues a new customer_id per order
seller	seller_id	zip_code_prefix → zip_code	location resolved through zip_code
category	category_name	—	Portuguese → English lookup (English unique)
product	product_id	category_name → category (nullable)	610 products have no category in the source
orders	order_id	customer_id → customer (UNIQUE)	1:1 with customer by Olist's design
shipment	(order_id, seller_id)	order_id → orders · seller_id → seller	one shipping deadline per seller per order
order_item	(order_id, product_id)	(order_id, seller_id) → shipment · product_id → product	an item's seller must ship in that order
payment	(order_id, payment_sequential)	order_id → orders	one order can be split (card + voucher)
review	review_id	—	score and text depend only on review_id
order_review	(order_id, review_id)	order_id → orders · review_id → review	M:N: 789 reviews cover several orders

[Dataset](#)[Schema](#)[SQL](#)[Next steps](#)

Normalization to 3NF, tested on the data

1NF

Atomic values, a key for every row

PROBLEM

geolocation has no candidate key:
1,000,163 rows, 261,831 exact duplicates.

FIX

zip_code: one row per zip (average of distinct points), zip stored as CHAR(5).

EVIDENCE

19,177 zips; parent table for customer and seller.

**2NF**

No partial dependencies

PROBLEM

reviews file are keyed by (review_id, order_id), yet review_id alone → score, title, message, dates.

FIX

split into review + bridge table
order_review.

EVIDENCE

789 shared ids, 0 FD violations; 98,410 reviews, 99,224 links.

**3NF**

No transitive dependencies

PROBLEM

customer → zip → state; (order, item #) → (order, seller) → deadline; item rows repeat seller, price.

FIX

state moves to zip_code; new shipment table; order_item gets quantity.

EVIDENCE

0 violations on 14,994 zips and 100,010 pairs; city stays (zip → city fails for 39 zips).

How we tested: `GROUP BY X HAVING COUNT(DISTINCT Y) > 1` → 0 rows means $X \rightarrow Y$ holds

[See the 4 tests ›](#)

[Dataset](#)[Schema](#)[SQL](#)[Next steps](#)

Did our data load correctly? 3 checks, all passed



9 raw CSV files

1,550,922 rows

LOAD + CLEAN WITH SQL

01 load → 02 schema → 03 clean → 04 verify



11 clean MySQL tables

758,133 rows, in 3NF

Fewer rows does not mean lost data: we removed duplicates. The geolocation file alone went from 1,000,163 rows to 19,177 zip codes.

1

Did every file load?

9 of 9

CSV files loaded with exactly the right number of rows. Example: orders 99,441 in the file = 99,441 in MySQL.

✓ Passed

2

Did cleaning lose anything?

Same totals

Before and after cleaning: 112,650 items sold and R\$13,591,643.70 revenue, equal to the cent.

✓ Passed

3

Do all table links work?

0 broken

No row points to something missing (0 orphan rows). MySQL checked every foreign key while loading.

✓ Passed

[Dataset](#)[Schema](#)[SQL](#)[Next steps](#)

Drumil · what is each customer's latest order?

ADVANCED 2

What is each customer's most recent order?

[All 4 + chart ›](#)

How it works

- 1 Inner query runs first**
For each person (customer_unique_id), MAX(purchase_ts) finds the date of their latest order.
- 2 Its result becomes table lo (latest orders)**
One row per person, named lo (latest order), so the outer query can join it like a table.
- 3 Join back on 2 columns**
Match person AND date, so only the order placed at that moment survives; MAX alone can't say which order.
- 4 Sort, limit, check ties**
Newest first, top 20. Same-second ties keep both orders: 96,367 rows for 96,096 people.

96,096

people, each with their latest order (+271 rows from same-second ties); newest 17 Oct 2018

1

2

3

4

```
SELECT c.customer_unique_id, o.order_id, o.purchase_ts
FROM customer c
JOIN orders o ON c.customer_id = o.customer_id
JOIN (SELECT c2.customer_unique_id,
            MAX(o2.purchase_ts) AS last_date
      FROM customer c2
      JOIN orders o2 ON c2.customer_id = o2.customer_id
      GROUP BY c2.customer_unique_id) lo
  ON c.customer_unique_id = lo.customer_unique_id
 AND o.purchase_ts = lo.last_date
ORDER BY o.purchase_ts DESC
LIMIT 20;
```

ALSO PRESENTED[BASIC 1](#) Orders ↔ customers: all matched[BASIC 2](#) Orders without a review: 768[ADVANCED 1](#) Spent > R\$500: 3,753 people

[Dataset](#)[Schema](#)[SQL](#)[Next steps](#)

Ellie · Product categories with at least 100,000 in revenue

ADVANCED 2

Product categories with at least 100,000 in revenue

[All 4 + chart ›](#)

How it works

- 1 Join 3 tables**
Start at order_item. Add product to get each item's category, then category for the English name.
- 2 Revenue per row**
The schema stores the price of one unit, so each line is quantity × unit price. For example, 3 items at R\$20 is R\$60.
- 3 GROUP BY category**
Items from the same category become one group. **SUM** adds up the revenue, so thousands of item rows become one row per category.
- 4 HAVING filters groups**
Keep only categories with total revenue of at least R\$100,000. **HAVING** is needed because the total only exists after grouping.

```
SELECT c.category_name_en AS category,
       ROUND(SUM(oi.quantity * oi.unit_price), 2) AS
revenue
FROM order_item oi
JOIN product p ON oi.product_id = p.product_id
JOIN category c ON p.category_name =
c.category_name
GROUP BY c.category_name_en
HAVING SUM(oi.quantity * oi.unit_price) >= 100000
ORDER BY revenue DESC;
```

26

categories earn at least R\$100,000 in revenue

ALSO PRESENTED

BASIC 1 Orders per status: 97% delivered**BASIC 2** Payment types: card R\$12.5M**ADVANCED 1** Categories ≥ R\$100k: 26

[Dataset](#)[Schema](#)[SQL](#)[Next steps](#)

Sathwika : 5-star delivered orders

ADVANCED 1

Which delivered orders got a 5-star review?

[All 4 + chart ›](#)

How it works

1

Join customer to orders

Each order picks up the buyer's city through customer_id.

2

Keep delivered orders

WHERE keeps only the 96,478 orders with status 'delivered'.

3

Correlated subquery

orv.order_id = o.order_id points at the outer row, so the subquery is checked once for each order.

4

EXISTS: yes or no

EXISTS only asks whether a 5-star review exists. A plain JOIN would return 57,066 rows (249 repeats).

```
1 SELECT c.customer_id, c.city, o.order_id,
   o.order_status
   FROM customer AS c
  INNER JOIN orders AS o
        ON o.customer_id = c.customer_id
 4 WHERE o.order_status = 'delivered'
      AND EXISTS (
          SELECT 1
         FROM order_review AS orv
        INNER JOIN review AS r
         3         ON r.review_id = orv.review_id
          WHERE orv.order_id = o.order_id
            AND r.score = 5)
   ORDER BY o.order_id;
```

56,817

delivered orders with a 5-star review, 58.9% of all deliveries

ALSO PRESENTED[BASIC 1](#) Products per category: top 3,029[BASIC 2](#) Deliveries by state: SP 42%[ADVANCED 2](#) Items above avg price: 29,775

[Dataset](#)[Schema](#)[SQL](#)[Next steps](#)

Aimaan : loyal, happy repeat buyers

ADVANCED 2

Which loyal buyers (3+ orders) never rated below 4 stars?

All 4 + chart ›

How it works

1 Loyal: IN + HAVING

Each customer row is one order, so HAVING COUNT(*) >= 3 per person lists the 252 people with 3+ orders.

2 Unhappy: NOT EXISTS

The second subquery walks customer → orders → order_review → review looking for any score below 4.

3 Correlated link

c2.customer_unique_id = c.customer_unique_id ties that search to the outer person, one person at a time.

4 Count and rank

GROUP BY person and COUNT(*) gives orders per person; the most loyal buyer placed 7.

```
1 SELECT c.customer_unique_id, z.state,
   COUNT(*) AS orders_placed
   FROM customer c
   JOIN zip_code z ON z.zip_code_prefix = c.zip_code_prefix
2 WHERE c.customer_unique_id IN (
   SELECT customer_unique_id FROM customer
   GROUP BY customer_unique_id
   HAVING COUNT(*) >= 3)
3 AND NOT EXISTS (
   SELECT 1
   FROM customer c2
   JOIN orders o ON o.customer_id = c2.customer_id
   JOIN order_review orv ON orv.order_id = o.order_id
   JOIN review r ON r.review_id = orv.review_id
4 WHERE c2.customer_unique_id = c.customer_unique_id
   AND r.score < 4)
5 GROUP BY c.customer_unique_id, z.state
6 ORDER BY orders_placed DESC;
```

162

of 252 repeat buyers never rated below 4 stars
(166 rows: 4 people ordered from two states)

ALSO PRESENTED

BASIC 1 Payments above avg: 31,012

BASIC 2 1-star customers: SP 3,965

ADVANCED 1 Sellers above avg: 628 of 3,095

[Dataset](#)[Schema](#)[SQL](#)[Next steps](#)

Issues we hit while querying

WHAT WENT WRONG

customer_id is not a person. Olist issues a new id per order, so customer LEFT JOIN orders never returns NULL.

AVG(SUM(...)) is rejected. MySQL cannot nest aggregate functions.

Grouping by (person, state) split people. 166 rows came back for 162 loyal buyers: 4 ordered from two states.

Unbounded results. The spender join returned 95,420 rows and the 5-star EXISTS query 56,817.

NULL timestamps. 1,783 orders never reached a carrier, and NULL > deadline is neither true nor false.

HOW WE FIXED IT



Count people with customer_unique_id: 96,096 people behind 99,441 ids, 2,997 repeat buyers.



Build per-seller totals in a derived table, then average them.



Count DISTINCT customer_unique_id, or group by the person only.



Let SQL answer the question: HAVING > R\$500 leaves 3,753 customers; LIMIT for top-N.



Filter delivered orders with delivered_carrier_ts IS NOT NULL before computing late %.

[Dataset](#)[Schema](#)[SQL](#)[Next steps](#)

Challenges so far and plan for the final

Major difficulties so far



Data cleaning

Free-text seller cities, " in dates and numbers, 0 g weights: cleaned or set to NULL, never deleted.



Schema mapping

Choosing the grain (unit rows → quantity) and modelling customer_id vs customer_unique_id.



FK enforcement

Checks ON means parents first: 2 missing categories added, 167 zips kept with NULL coordinates.



Performance & import

1M-row geolocation step is the slowest, CRLF, BOM and “\” traps in LOAD DATA.

Planned for the final

1

Indexes + EXPLAIN

EXPLAIN the ~1.4 s joins, index order_item(seller_id) and review(score), report before/after times.

2

Advanced queries

CTEs, window functions (RANK, LAG), views, and delivery delay vs review score.

3

Dashboard

Streamlit + Plotly on MySQL: 2+ charts per member, filters by state, category and month.

4

Report & repo

README with run order, saved results, final ER + schema, FD-test script.

Thank you. Questions?

WHO DID WHAT · FROM OUR GITHUB HISTORY (21 COMMITS, ALL 4 MEMBERS)

Sathwika Roy Tulasi

Raw CSVs, 01_load.sql, subqueries_joins.sql

Aimaan Khan

02_schema.sql (PKs, FKs, CHECKs), subqueries.sql

Drumil PawarPatil

03_Cleaning.sql, joins.sql

Ellie Le

ER diagram, 04_verify.sql, aggregation.sql



Open the GitHub repo



Q&A backup: full SQL



Back to the agenda

Repo: github.com/SathwikaRoy2002/BrazilianE-Commerce_group7_data201

Q&A backup: full SQL and evidence

El

aggregation.sql

[All 4 + chart ›](#)

[Basic queries ›](#)

[Advanced queries ›](#)

Drumil

joins.sql

[All 4 + chart ›](#)

[Basic queries ›](#)

[Advanced queries ›](#)

Sathwika

subqueries_joins.sql

[All 4 + chart ›](#)

[Basic queries ›](#)

[Advanced queries ›](#)

Aimaan

subqueries.sql

[All 4 + chart ›](#)

[Basic queries ›](#)

[Advanced queries ›](#)



Functional-dependency tests ›

The 4 GROUP BY ... HAVING checks behind our 2NF and 3NF decisions



Load evidence ›

Terminal run of 01_load.sql and a 6-table sample join from 04_verify.sql

[Back to the closing slide](#)

El · Aggregation: revenue and late sellers

BASIC 1

[SQL ›](#)

How many orders are in each status?

GROUP BY · COUNT · ORDER BY

8 statuses; 96,478 delivered (97.0%), 625 canceled

BASIC 2

[SQL ›](#)

Which payment types bring in the most money?

GROUP BY · SUM · AVG · ROUND

Credit card: 76,795 payments, R\$12.54M, avg R\$163.32

ADVANCED 1

[SQL ›](#)

Which categories earn at least R\$100k?

4-table JOIN · WHERE · GROUP BY · HAVING

26 categories; health_beauty leads with R\$1.26M

ADVANCED 2

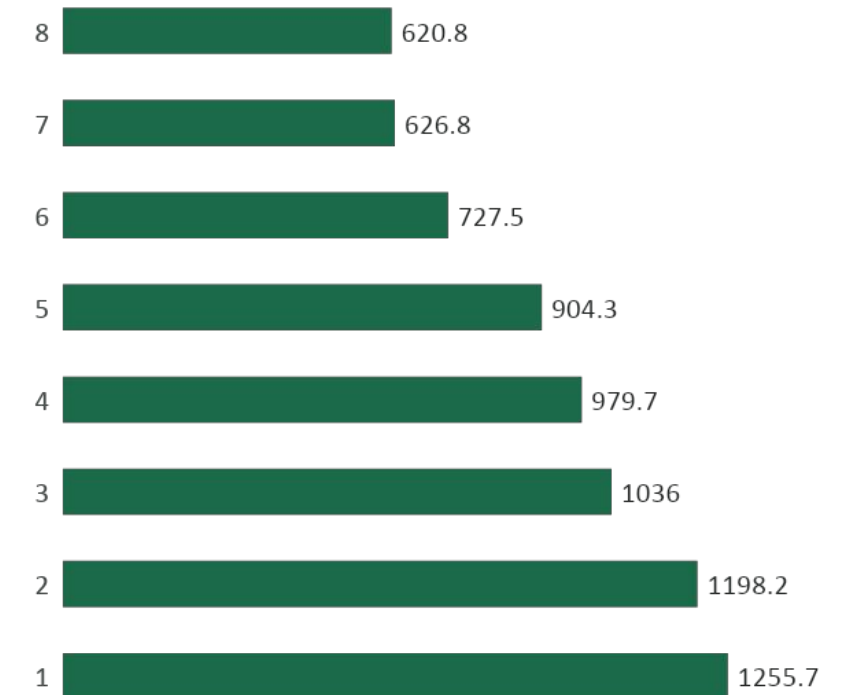
[SQL ›](#)

Which sellers ship late too often?

3 JOINS · HAVING on AVG(boolean) · LIMIT

188 of 627 busy sellers are late >10% of the time; worst 84.8%

Revenue of the top 8 categories (R\$ thousands)



Advanced 1 output, canceled/unavailable excluded

Drumil · Joins: orders, customers, reviews

BASIC 1

[SQL ›](#)

Which customer placed each order?

INNER JOIN on customer_id

All 99,441 orders match; first one 4 Sep 2016 (Boa Vista, RR)

BASIC 2

[SQL ›](#)

Which orders never got a review?

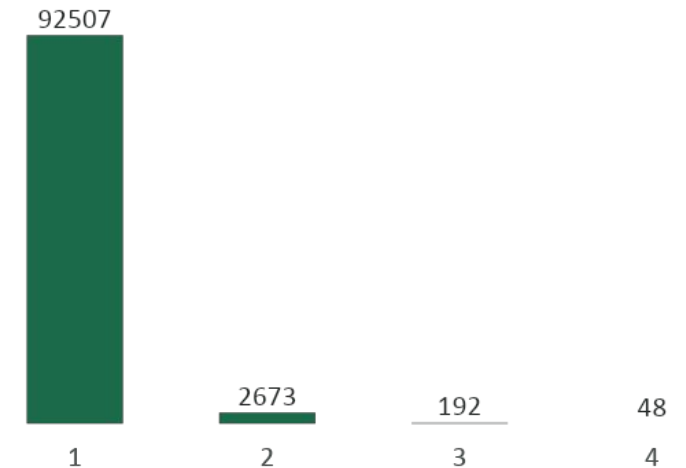
LEFT JOIN · WHERE ... IS NULL

768 orders, 646 of them delivered

96.9%

of customers bought only once (by customer_unique_id)

Customers by number of orders placed



Advanced 1 join before the HAVING filter

ADVANCED 1

[SQL ›](#)

Who spent more than R\$500 in total?

3-table JOIN · GROUP BY person · HAVING

3,753 customers; top spender R\$13,440 in one order

ADVANCED 2

[SQL ›](#)

What is each customer's most recent order?

JOIN to a derived table with MAX()

One latest order per person; newest is 17 Oct 2018

Sathwika · Joins with subqueries

BASIC 1

[SQL ›](#)

Which categories have the most products?

LEFT JOIN · COUNT · GROUP BY

bed_bath_table 3,029 · sports_leisure
2,867

BASIC 2

[SQL ›](#)

Which states receive the most deliveries?

3-table JOIN · WHERE · GROUP BY

27 states; SP takes 42% (40,501 orders)

ADVANCED 1

[SQL ›](#)

Which delivered orders got a 5-star review?

JOIN + correlated EXISTS subquery

56,817 orders, 58.9% of all deliveries

ADVANCED 2

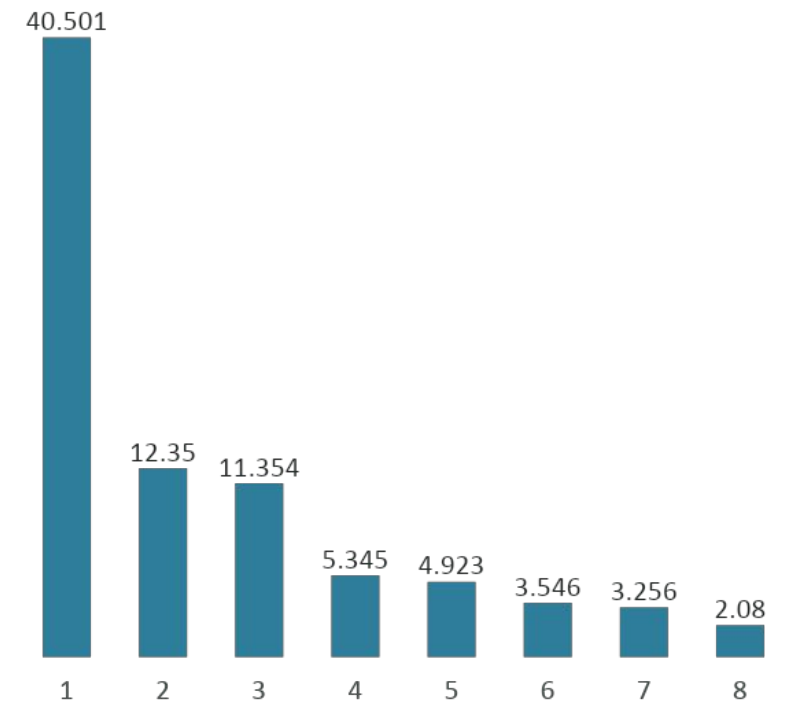
[SQL ›](#)

Which items cost more than the average item?

JOIN + scalar subquery (AVG)

29,775 of 102,425 item lines above
R\$124.42

Delivered orders by state, top 8 (thousands)



Basic 2 output · 27 states in total

Aimaan · Subqueries, scalar to correlated

BASIC 1

[SQL ›](#)

Which payments are above the average payment?

Scalar subquery in WHERE

31,012 of 103,886 above R\$154.10; max R\$13,664

BASIC 2

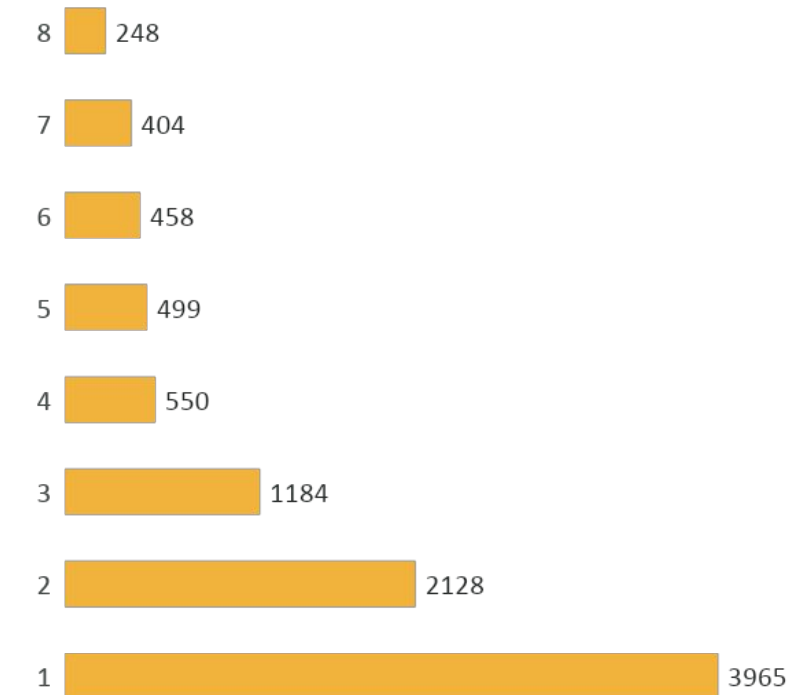
[SQL ›](#)

Which states have the most 1-star customers?

3-level nested IN subqueries

SP 3,965 · RJ 2,128 · MG 1,184

Customers who left a 1-star review, top 8 states



Basic 2 output · distinct customer_unique_id

ADVANCED 1

[SQL ›](#)

Which sellers earn more than the average seller?

Derived table + nested scalar subquery

628 of 3,095 sellers; #1 earns R\$229k (Guariba, SP)

ADVANCED 2

[SQL ›](#)

Which loyal buyers (3+ orders) never rated below 4 stars?

IN + HAVING + correlated NOT EXISTS

162 customers (166 rows); top buyer placed 7 orders

El · basic queries (full SQL)

BASIC 1

How many orders are in each status?

```
SELECT order_status,  
       COUNT(*) AS order_count  
FROM   orders  
GROUP BY order_status  
ORDER BY order_count DESC;
```

Result 8 rows · delivered 96,478 · shipped 1,107 · canceled 625 · unavailable 609

BASIC 2

Which payment types bring in the most money?

```
SELECT payment_type,  
       COUNT(*) AS payment_count,  
       ROUND(SUM(payment_value), 2) AS total_value,  
       ROUND(AVG(payment_value), 2) AS avg_value  
FROM   payment  
GROUP BY payment_type  
ORDER BY total_value DESC;
```

Result 5 rows · credit_card R\$12,542,084.19 · boleto R\$2,869,361.27 · voucher R\$379,436.87

El - advanced queries (full SQL)

ADVANCED 1

Which categories earn at least R\$100k?

```
SELECT c.category_name_en AS category,
       COUNT(DISTINCT o.order_id) AS order_count,
       SUM(oi.quantity) AS units_sold,
       ROUND(SUM(oi.quantity * oi.unit_price), 2)
         AS revenue
FROM   order_item oi
JOIN   orders o ON oi.order_id = o.order_id
JOIN   product p ON oi.product_id = p.product_id
JOIN   category c ON p.category_name = c.category_name
WHERE  o.order_status NOT IN ('canceled', 'unavailable')
GROUP BY c.category_name, c.category_name_en
HAVING SUM(oi.quantity * oi.unit_price) >= 100000
ORDER BY revenue DESC;
```

Result 26 rows · health_beauty R\$1,255,695 · watches_gifts R\$1,198,185 · bed_bath_table R\$1,035,964

ADVANCED 2

Which sellers ship late too often?

```
SELECT s.seller_id, z.state,
       COUNT(*) AS shipments,
       SUM(o.delivered_carrier_ts > sh.shipping_limit_ts)
         AS late_shipments,
       ROUND(100 * AVG(o.delivered_carrier_ts
                       > sh.shipping_limit_ts), 1) AS late_pct
FROM   shipment sh
JOIN   orders o ON sh.order_id = o.order_id
JOIN   seller s ON sh.seller_id = s.seller_id
JOIN   zip_code z ON s.zip_code_prefix = z.zip_code_prefix
WHERE  o.order_status = 'delivered'
       AND o.delivered_carrier_ts IS NOT NULL
GROUP BY s.seller_id, z.state
HAVING COUNT(*) >= 30
       AND AVG(o.delivered_carrier_ts
               > sh.shipping_limit_ts) > 0.10
ORDER BY late_pct DESC
LIMIT 10;
```

Result Top 10 of 188 · worst seller (PR) late on 28 of 33 shipments = 84.8%

Drumil · basic queries (full SQL)

BASIC 1

Which customer placed each order?

```
SELECT o.order_id, o.purchase_ts,  
       c.customer_unique_id, c.city  
FROM   orders o  
INNER JOIN customer c  
       ON o.customer_id = c.customer_id  
ORDER BY o.purchase_ts  
LIMIT 20;
```

Result 20 rows · earliest order 2016-09-04 21:15 (boa vista) · no names in the data, so customer_unique_id + city identify the buyer

BASIC 2

Which orders never got a review?

```
SELECT o.order_id, o.order_status, orv.review_id  
FROM   orders o  
LEFT JOIN order_review orv  
       ON o.order_id = orv.order_id  
WHERE  orv.review_id IS NULL;
```

Result 768 rows · delivered 646 · shipped 75 · canceled 20 · unavailable 14

Drumil · advanced queries (full SQL)

ADVANCED 1

Who spent more than R\$500 in total?

```
SELECT c.customer_unique_id,
       COUNT(DISTINCT o.order_id) AS orders_placed,
       SUM(oi.quantity * oi.unit_price) AS total_spent
FROM   customer c
JOIN   orders o      ON c.customer_id = o.customer_id
JOIN   order_item oi  ON o.order_id    = oi.order_id
GROUP BY c.customer_unique_id
HAVING SUM(oi.quantity * oi.unit_price) > 500
ORDER BY total_spent DESC;
```

Result 3,753 rows (95,420 without HAVING) · top spender R\$13,440.00 in a single order

ADVANCED 2

What is each customer's most recent order?

```
SELECT c.customer_unique_id, o.order_id, o.purchase_ts
FROM   customer c
JOIN   orders o  ON c.customer_id = o.customer_id
JOIN   (SELECT c2.customer_unique_id,
              MAX(o2.purchase_ts) AS last_date
        FROM   customer c2
        JOIN   orders o2 ON c2.customer_id = o2.customer_id
        GROUP BY c2.customer_unique_id) lo
      ON c.customer_unique_id = lo.customer_unique_id
      AND o.purchase_ts       = lo.last_date
ORDER BY o.purchase_ts DESC
LIMIT 20;
```

Result 20 rows · newest order 2018-10-17 17:30 · derived table lo = one row per person

Sathwika · basic queries (full SQL)

BASIC 1

Which categories have the most products?

```
SELECT c.category_name_en AS category,
       COUNT(p.product_id) AS product_count
FROM   category AS c
LEFT JOIN product AS p
      ON p.category_name = c.category_name
GROUP BY c.category_name, c.category_name_en
ORDER BY product_count DESC, category
LIMIT 10;
```

Result 10 rows · bed_bath_table 3,029 · sports_leisure 2,867 · furniture_decor 2,657

BASIC 2

Which states receive the most deliveries?

```
SELECT z.state,
       COUNT(*) AS delivered_order_count
FROM   orders AS o
INNER JOIN customer AS c
      ON c.customer_id = o.customer_id
INNER JOIN zip_code AS z
      ON z.zip_code_prefix = c.zip_code_prefix
WHERE  o.order_status = 'delivered'
GROUP BY z.state
ORDER BY delivered_order_count DESC, z.state;
```

Result 27 rows · SP 40,501 · RJ 12,350 · MG 11,354 · RS 5,345

Sathwika · advanced queries (full SQL)

ADVANCED 1

Which delivered orders got a 5-star review?

```
SELECT c.customer_id, c.city, o.order_id, o.order_status
FROM   customer AS c
INNER JOIN orders AS o
      ON o.customer_id = c.customer_id
WHERE  o.order_status = 'delivered'
      AND EXISTS (
        SELECT 1
        FROM    order_review AS orv
        INNER JOIN review AS r
              ON r.review_id = orv.review_id
        WHERE   orv.order_id = o.order_id
              AND r.score = 5)
ORDER BY o.order_id;
```

Result 56,817 rows = 58.9% of the 96,478 delivered orders

ADVANCED 2

Which items cost more than the average item?

```
SELECT s.seller_id, s.city AS seller_city,
       i.product_id, i.unit_price
FROM   seller AS s
INNER JOIN order_item AS i
      ON i.seller_id = s.seller_id
WHERE  i.unit_price > (SELECT AVG(unit_price)
                      FROM order_item)
ORDER BY i.unit_price DESC;
```

Result 29,775 rows · average unit price R\$124.42 · most expensive line R\$6,735.00 (sao paulo)

Aimaan · basic queries (full SQL)

BASIC 1

Which payments are above the average payment?

```
SELECT order_id, payment_type,
       installments, payment_value
FROM   payment
WHERE  payment_value > ( SELECT AVG(payment_value)
                        FROM   payment)
ORDER BY payment_value DESC
LIMIT 10;
```

Result Top 10 of 31,012 · average R\$154.10 · largest R\$13,664.08 by credit card

BASIC 2

Which states have the most 1-star customers?

```
SELECT z.state,
       COUNT(DISTINCT c.customer_unique_id)
       AS unhappy_customers
FROM   customer c
JOIN   zip_code z ON z.zip_code_prefix = c.zip_code_prefix
WHERE  c.customer_id IN (
      SELECT o.customer_id FROM orders o
      WHERE o.order_id IN (
            SELECT orv.order_id FROM order_review orv
            WHERE orv.review_id IN (
                  SELECT review_id FROM review
                  WHERE score = 1)))
GROUP BY z.state
ORDER BY unhappy_customers DESC;
```

Result 27 rows · SP 3,965 · RJ 2,128 · MG 1,184 · RS 550

Aimaan · advanced queries (full SQL)

ADVANCED 1

Which sellers earn more than the average seller?

```
SELECT s.seller_id, s.city, z.state,
       t.orders_sold, t.revenue
FROM (SELECT seller_id,
             COUNT(DISTINCT order_id) AS orders_sold,
             SUM(quantity * unit_price) AS revenue
      FROM order_item
      GROUP BY seller_id) t
JOIN seller s ON s.seller_id = t.seller_id
JOIN zip_code z ON z.zip_code_prefix = s.zip_code_prefix
WHERE t.revenue > (SELECT AVG(seller_rev)
                  FROM (SELECT SUM(quantity * unit_price)
                        AS seller_rev
                        FROM order_item
                        GROUP BY seller_id) x)

ORDER BY t.revenue DESC
LIMIT 10;
```

Result Top 10 of 628 · #1 guariba (SP) R\$229,472.63 from 1,132 orders

ADVANCED 2

Which loyal buyers (3+ orders) never rated below 4 stars?

```
SELECT c.customer_unique_id, z.state,
       COUNT(*) AS orders_placed
FROM customer c
JOIN zip_code z ON z.zip_code_prefix = c.zip_code_prefix
WHERE c.customer_unique_id IN (
    SELECT customer_unique_id FROM customer
    GROUP BY customer_unique_id
    HAVING COUNT(*) >= 3)
AND NOT EXISTS (
    SELECT 1
    FROM customer c2
    JOIN orders o ON o.customer_id = c2.customer_id
    JOIN order_review orv ON orv.order_id = o.order_id
    JOIN review r ON r.review_id = orv.review_id
    WHERE c2.customer_unique_id = c.customer_unique_id
    AND r.score < 4)

GROUP BY c.customer_unique_id, z.state
ORDER BY orders_placed DESC;
```

Result 166 rows for 162 people (4 ordered from two states) · top: 7 orders, no review under 4 stars

Functional-dependency tests on the raw data

3NF · zip_code_prefix → state

```
SELECT customer_zip_code_prefix
FROM   stg_customers
GROUP  BY customer_zip_code_prefix
HAVING COUNT(DISTINCT customer_state) > 1;
```

✓ 0 of 14,994 zips → state moves to zip_code

3NF · (order_id, product_id) → seller, price, freight

```
SELECT order_id, product_id
FROM   stg_order_items
GROUP  BY order_id, product_id
HAVING COUNT(DISTINCT seller_id) > 1
       OR COUNT(DISTINCT price) > 1
       OR COUNT(DISTINCT freight_value) > 1;
```

✓ 0 rows → order_item with quantity

2NF · review_id → score, comment

```
SELECT review_id
FROM   stg_reviews
GROUP  BY review_id
HAVING COUNT(DISTINCT review_score) > 1
       OR COUNT(DISTINCT review_comment_message) > 1;
```

✓ 0 of 789 shared ids → review + order_review

3NF · (order_id, seller_id) → shipping_limit_date

```
SELECT order_id, seller_id
FROM   stg_order_items
GROUP  BY order_id, seller_id
HAVING COUNT(DISTINCT shipping_limit_date) > 1;
```

✓ 0 of 100,010 pairs → shipment table

Counter-example: zip → city fails for 39 customer zips, so city is not dependent on the zip and stays in customer. These tests run on the staging (stg_*) tables before cleaning.

Load evidence: terminal run and sample join

```
BrazilianE-Commerce_group7_data201 — -zsh — 80x24

(base) sathwikaroy@Sathwikas-MacBook-Pro BrazilianE-Commerce_group7_data201 % ls
data      sql
(base) sathwikaroy@Sathwikas-MacBook-Pro BrazilianE-Commerce_group7_data201 % /usr/local/mysql/bin/mysql
ERROR 1045 (28000): Access denied for user 'sathwikaroy'@'localhost' (using password: NO)
(base) sathwikaroy@Sathwikas-MacBook-Pro BrazilianE-Commerce_group7_data201 % ls
/usr/local/mysql/bin/mysql
/usr/local/mysql/bin/mysql
(base) sathwikaroy@Sathwikas-MacBook-Pro BrazilianE-Commerce_group7_data201 % /usr/local/mysql/bin/mysql --local-infile=1 -u root -p < sql/01_load.sql
Enter password:
tbl      n
stg_customers  99441
stg_geolocation 1000163
stg_order_items 112650
stg_payments    103886
stg_reviews     99224
stg_orders      99441
stg_products    32951
stg_sellers     3095
stg_category_translation 71
(base) sathwikaroy@Sathwikas-MacBook-Pro BrazilianE-Commerce_group7_data201 %
```

docs/data_loading.png · mysql --local-infile=1 -u root -p < sql/01_load.sql; staging counts match all 9 source files

Sample 6-table join (04_verify.sql, V6)

```
SELECT o.order_id, z.state, c.city, cat.category_name_en, ...
FROM orders o JOIN customer c ON ... JOIN zip_code z ON ...
JOIN order_item i ON ... JOIN product p ON ...
JOIN category cat ON ...
ORDER BY o.purchase_ts LIMIT 5;
```

order_id	City, state	Category	Qty	Price	Status
2e7a8482...	boa vista, RR	furniture_decor	1	32.90	shipped
2e7a8482...	boa vista, RR	furniture_decor	1	39.99	shipped
e5fa5a72...	passo fundo, RS	telephony	1	59.50	canceled
bfb0f9b...	sao joaquim da barra, SP	health_beauty	3	44.99	delivered
71303d7e...	sao paulo, SP	baby	1	100.00	canceled

Every order resolves through customer → zip_code (state) and order_item → product → category: the FKs work end to end.